Optimisation of a braking system

Alexis JENBACK - Pape Mor NIANG

```
clear all; close all; clc;
%
%% Partie 1
absdata;
P_vec = [0:1:10];     % Gain proportionnel
K_vec = [0:0.1:1];    % Gain derive
cons=0.2;
for i=1:length(P_vec)
    for j=1:length(K_vec)
        P=P_vec(i);
        Ki=K_vec(j);
        sim('ABS_PD.slx')
        distance_vec(i,j)=Distance;
    end
end
%% figures
figure(1),
contourf(P_vec,K_vec,distance_vec')
colorbar
xlabel('$P$','Interpreter','latex');
ylabel('$D$','Interpreter','latex');
set(gcf,'Color','w','Position',[25 100 600 600]);
set(gca,'fontname','times','fontsize',16);
legend('$Distance [m]$','Interpreter','latex','location','north outside
saveas(gcf,'Q1figure1','jpg')
figure(2),
surf(P_vec,K_vec,distance_vec')
xlabel('P');ylabel('D');zlabel('distance [m]')
xlabel('$P$','Interpreter','latex');
ylabel('$D$','Interpreter','latex');
zlabel('$Distance [m]$','Interpreter','latex');
set(gcf,'Color','w','Position',[25 100 600 600]);
set(gca,'fontname','times','fontsize',16);
legend('$Distance [m]$','Interpreter','latex','location','north outside
saveas(gcf,'Q1figure2','jpg')
%% Distance minimum
[m,n]=size(distance_vec);
min = distance_vec(1,1);
for i = 1:m
    for j = 1:n
        if min > distance_vec(i,j)
            min = distance_vec(i,j);
            ligne = i;
            colonne = j;
        end
    end
end
% Sur la figure(2), la distance de freinage est minimale lorsque la val
% et celle du gain D de 0.3 [D_vec(colonne)]
```

```
%% Question 2
close; clc;clear all;
absdata;

epsilon=1;
delta_x=0;
delta_p=0.1;
delta_k=0.1;

P1=5;
Ki1=0.3;
mu = [0.049 0.05] %[0.999 1.0] %[0.719 0.72]; %[0.5099 0.51]  %[0.2799 (
slip =[0.01 0.02]  %[0.0799 0.08] %[0.0599 0.06];  %[0.051 0.052] %[0.0
    cons=1;
    for s=1:length(vitesse)
        P=P1;
        Ki=Ki1;
        k=1;
    %for a=1:5
        %k=1;
        %if a ~= 1
        %mu = mu + 0.23;%
        %slip = slip +0.016;
        %end
    while k<5 && dx<epsilon

        gP=[P,P+delta_p,P-delta_p];
        gK=[Ki,Ki+dk,Ki-delta_k];
        D=ones(length(gP));
        v0=vitesse(s)/3.6;  %50/3.6; %70/3.6; %90/3.6; %110/3.6; 130/3.(

        for i=1:length(gP)
            P=gP(i);
            for j=1:length(gK)
                Ki=gK(j);
                sim("ABS_PD.slx")
                D(i,j)=Distance; % la fonction distance
                D2(i,j)=D(i,j)^2; % la fonction Cost
            end
        end
        Gradient=[HesP(D2,delta_p);HesD(D2,delta_k)]; % calcul du gradi(
        H=[Hes2P(D2,delta_p) HesPD(D2,delta_p,delta_k);HesPD(D2,dp,delt(

        d=-H^-1*Gradient; % direction de descente

        P=P+delta_p*d(1); % nouvelle valeur de P
        Ki=Ki+delta_k*d(2); % nouvelle valeur de K

        dx=(D(3,3)-D(2,3))*100;

        k=k+1;
    end
    G_P(1,s)=P; %
    G_K(1,s)=Ki;
    distance(1,s)=Distance;
    %end
```

```
    end
%% Résultats matrice mu, P et D
mu_vec = [0.05 0.28 0.51 0.72 1 ];
G_P_vec=[4.7295 4.9281 4.7125 4.9866 5.2427 ;
 5.0480 5.0170 4.9110 4.6984 4.7240 ;
 5.5039 5.6864 5.3102 5.2909 4.6040 ;
 5.8831 5.4578 4.9592 4.6018 4.6380 ;
 6.1981 5.5450 5.4814 5.4111 5.4532 ];
G_K_vec=[-0.1259 -0.1116 -0.1058 -0.1175 -0.1053 ;
 -0.1022 -0.1027 -0.1097 -0.1057 -0.1076 ;
 -0.0750 -0.0643 -0.1275 -0.1242 -0.1071 ;
 -0.0491 -0.0764 -0.1116 -0.1072 -0.1095 ;
 -0.0260 -0.0704 -0.0733 -0.0724 -0.0762 ];
%% figures
figure,
surf(vitesse,mu_vec,G_P_vec)
xlabel('V [km/h]');ylabel('mu');zlabel('P')
xlabel('$V [km/h]$','Interpreter','latex');
ylabel('$mu$','Interpreter','latex');
zlabel('$P$','Interpreter','latex');
set(gcf,'Color','w','Position',[25 100 600 600]);
set(gca,'fontname','times','fontsize',16);
legend('$P$','Interpreter','latex','location','north outside');
saveas(gcf,'Q2figure1','jpg')

figure,
surf(vitesse,mu_vec,G_K_vec)
xlabel('V [km/h]');ylabel('mu');zlabel('P')
xlabel('$V [km/h]$','Interpreter','latex');
ylabel('$mu$','Interpreter','latex');
zlabel('$D$','Interpreter','latex');
set(gcf,'Color','w','Position',[25 100 600 600]);
set(gca,'fontname','times','fontsize',16);
legend('$D$','Interpreter','latex','location','north outside');
saveas(gcf,'Q2figure2','jpg')

%% Gradient et Hessien
```

```
function y=HesP(M,d)
y=(M(2,1)-M(1,1))/(2*d);
end

function y=HesD(M,d)
y=(M(1,2)-M(1,1))/(2*d);
end

function y=Hes2P(M,d)
y=(M(2,1)-2*M(1,1)+M(3,1))/(d^2);
end

function y=Hes2D(M,d)
y=(M(1,2)-2*M(1,1)+M(1,3))/(d^2);
end

function y=HesPD(M,d,k)
y=(M(2,2)-M(3,2)+M(3,3)-M(2,3))/(4*d*k);
```

```
end
```